

**LAPORAN AKHIR UAS KEAMANAN DAN INTEGRITAS DATA
“LAYANAN KRIPTOGRAFI API”**



Disusun Oleh:

Nor Hofifah	24031554009
Zahra Brillianty Putri	24031554014
Khofifah Dwi Sanni El Randi	24031554156

Dosen Pengampu:

Ibnu Febry Kurniawan, S.Kom., M.Sc.

**PROGRAM STUDI S1 SAINS DATA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS NEGERI SURABAYA**

2025

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam zaman digital yang didorong oleh pertukaran data yang besar, keamanan dan integritas informasi menjadi pondasi penting dalam komunikasi saat ini. Kriptografi berfungsi sebagai salah satu alat utama untuk melindungi data dari berbagai ancaman seperti penyadapan, modifikasi, dan pemalsuan identitas. Di sisi lain, Application Programming Interface (API) telah menjadi elemen kunci dalam integrasi sistem, memungkinkan berbagai aplikasi untuk saling berinteraksi dan bertukar data dengan cara yang efisien. Menurut Amazon Web Services (AWS), API adalah sebuah metode yang memungkinkan dua bagian perangkat lunak berkomunikasi menggunakan sekumpulan definisi dan protokol yang telah ditetapkan.

Penggabungan antara layanan API dan teknik kriptografi memberikan peluang untuk membangun sistem yang tidak hanya berfungsi dengan baik, tetapi juga tahan terhadap serangan keamanan. Sistem seperti ini dapat bertindak sebagai pihak ketiga yang dipercaya untuk memverifikasi dan melindungi pertukaran data. Dalam konteks akademis, pemahaman yang mendalam mengenai implementasi praktis dari konsep-konsep ini merupakan keterampilan yang sangat penting bagi para calon profesional di bidang sains data dan keamanan siber.

Evaluasi ini dirancang berdasarkan skenario **Punk Records-v1**, sebuah sistem yang mereplikasi lingkungan laboratorium canggih yang dimiliki Vegapunk di pulau Egghead. Sistem ini membutuhkan alat untuk menyimpan public key peneliti, memeriksa tanda tangan digital pada pesan, serta melakukan pengiriman pesan yang aman antar pengguna. Melalui proyek ini, kerangka kerja FastAPI serta algoritma kriptografi modern seperti EC-SECP256K1 dan ED25519 dengan menerapkan solusi keamanan yang berbasis API.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, masalah yang ingin diatasi dalam evaluasi dapat dirumuskan sebagai berikut:

1. Bagaimana cara merancang dan mengimplementasikan layanan API sederhana yang dapat menyimpan serta mengelola public key dari beberapa pengguna (peneliti) secara aman?
2. Bagaimana cara membangun mekanisme verifikasi tanda tangan digital untuk menjamin keaslian dan integritas pesan yang dipertukarkan dalam sistem?
3. Bagaimana cara membuat sistem pengalihan (relay) pesan yang aman antara dua pengguna dengan memanfaatkan teknik kriptografi asimetris maupun simetris?

4. Bagaimana cara meningkatkan keandalan sistem dengan menambahkan fitur-fitur opsional seperti dukungan multiuser, penggunaan variasi algoritma kunci, dan penerapan sesi yang aman menggunakan token?

1.3 Manfaat

Evaluasi diharapkan dapat memberikan manfaat baik secara teoritis maupun praktis:

1. Meningkatkan pengetahuan konseptual mengenai penggabungan kriptografi dalam layanan API.
2. Mengembangkan kemampuan teknis dalam menggunakan kerangka kerja FastAPI dan pustaka kriptografi (cryptography).
3. Memberikan contoh implementasi yang dapat diperluas untuk tujuan penelitian atau proyek lanjutan.
4. Memberikan pemahaman mengenai signifikansi verifikasi tanda tangan dan enkripsi end-to-end dalam pertukaran data sensitif.

1.4 Tujuan

Tujuan dari evaluasi ini adalah menerapkan sistem **Punk Records-v1** sebagai *trusted authority server* miniatur yang mengintegrasikan layanan kriptografi melalui API. Adapun tujuan spesifik yang ingin dicapai:

1. Membangun Layanan Penyimpanan Public Key:

Menciptakan endpoint API yang dapat menerima serta menyimpan public key peneliti dengan cara yang aman dan terstruktur.

2. Mengimplementasikan Verifikasi Tanda Tangan Digital:

Menyusun fungsi untuk memeriksa keaslian pesan dengan menggunakan pasangan kunci privat dan publik serta algoritma tanda tangan seperti ED25519 atau ECDSA.

3. Mendesain Sistem Relay Pesan Aman:

Membuat mekanisme untuk mengirim pesan antar dua peneliti dengan proses enkripsi dan dekripsi yang menjamin kerahasiaan dan integritas data.

4. Mengembangkan Fitur Tambahan (Opsional):

Menambahkan kemampuan seperti dukungan banyak pengguna, adopsi berbagai algoritma kriptografi (simetris/asimetris), serta penerapan mekanisme autentikasi sesi dengan menggunakan JWT atau token Bearer.

5. Mengevaluasi Kinerja dan Keamanan Sistem:

Melaksanakan uji coba fungsional serta analisis keamanan terhadap implementasi yang telah dibangun, serta mendokumentasikan proses dan hasilnya secara komprehensif.

BAB II

DESKRIPSI PROGRAM

2.1 Deskripsi Umum Program

Program Punk Records-v1 merupakan sebuah implementasi layanan kriptografi berbasis API, yang dibangun dengan menggunakan framework FastAPI. Program ini berfungsi sebagai trusted authority server yang bertanggung jawab untuk memastikan keamanan, integritas, dan keaslian data selama pertukaran informasi antar pengguna (peneliti Egghead).

Aplikasi ini menerapkan mekanisme kriptografi baik asimetris maupun simetris untuk mendukung proses autentikasi, verifikasi tanda tangan digital, serta pengiriman pesan dengan aman. Desain sistem memungkinkan akses dari client melalui endpoint API dengan menggunakan protokol HTTP.

2.2 Tujuan Pengembangan

Pengembangan program Punk Records-v1 memiliki tujuan untuk:

1. Memberikan layanan penyimpanan public key pengguna secara terpusat.
2. Melaksanakan verifikasi tanda tangan untuk memastikan integritas dan keaslian pesan serta dokumen PDF.
3. Menyediakan cara untuk relay pesan dengan aman di antara pengguna dengan menggunakan enkripsi.
4. Mensimulasikan penerapan konsep secure session melalui penggunaan token.

2.3 Arsitektur Sistem

Sistem ini terdiri dari dua elemen utama:

1. Server (FastAPI)

Berperan sebagai trusted authority yang menangani proses autentikasi, penyimpanan public key, verifikasi tanda tangan, dan relay pesan dengan aman.

2. Client

Berfungsi untuk merepresentasikan pengguna. Client memiliki tanggung jawab untuk menghasilkan pasangan private-public key, melakukan penandatanganan pada pesan atau dokumen PDF, serta mengirimkan permintaan ke server melalui endpoint API.

BAB III

PENGUNAAN PROGRAM

Menjelaskan langkah-langkah penggunaan aplikasi Punk Records-v1 dari menghidupkan server hingga melakukan verifikasi tanda tangan digital pada file PDF. Tahapan langkah ini disusun mengikuti urutan implementasi:

3.1 Menjalankan Server API

Langkah pertama adalah menjalankan server FastAPI yang berfungsi sebagai *trusted authority*. Server diaktifkan melalui terminal di dalam direktori proyek menggunakan perintah berikut:

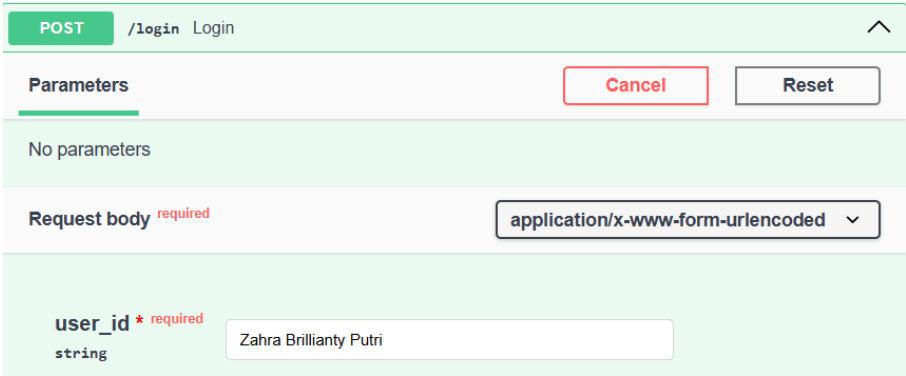
```
python -m uvicorn api:app --reload
```

Setelah server berhasil aktif, akan muncul informasi bahwa layanan berjalan di alamat: `http://127. 0. 0. 1:8000` dan kami mengakses Swagger UI melalui: `http://127. 0. 0. 1:8000/docs`.

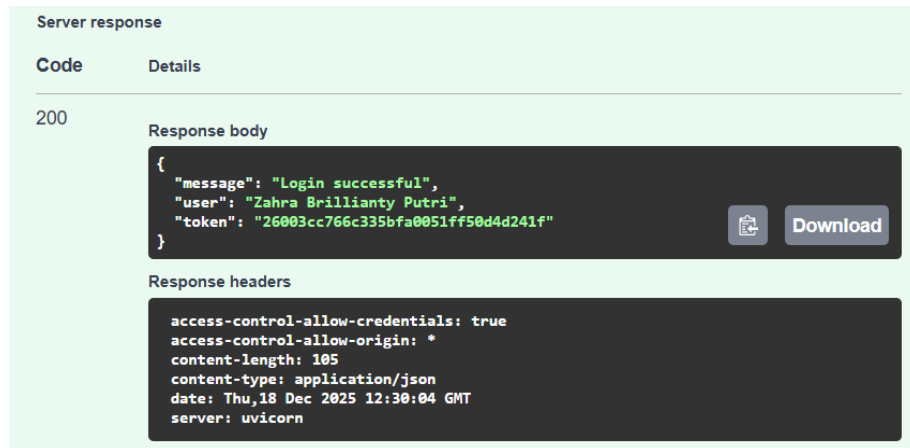
Swagger UI digunakan untuk melakukan pengujian endpoint secara langsung dan untuk melihat respons dari server.

3.2 Proses Login Pengguna

Sebelum mengakses layanan kriptografi, pengguna perlu masuk terlebih dahulu (login) untuk mendapatkan token sesi. Proses login ini kami lakukan melalui endpoint `/login` dengan mengirimkan `user_id`. Server kemudian akan membuat token spesifik yang berfungsi sebagai tanda autentikasi dalam langkah-langkah berikutnya.



Gambar diatas menampilkan antarmuka pengujian endpoint `/login` di Swagger UI. Pada gambar tersebut, terlihat bahwa pengguna memasukkan `user_id` nya. Langkah ini adalah langkah awal di mana client mengidentifikasi dirinya kepada server sebelum mengakses layanan kriptografi yang dilindungi.



Gambar diatas merupakan respons server setelah permintaan login berhasil diproses. Server memberikan respons berupa kode status HTTP 200 (OK) yang menunjukkan bahwa prosesnya berhasil dan token sesi dengan nilai "24883cc7666335bfa8051f758d4d241f". Token unik ini berfungsi sebagai kunci autentikasi untuk semua permintaan API berikutnya. Header dari respons juga menunjukkan bahwa server beroperasi menggunakan Uvicorn.

3.3 Penyimpanan Public Key

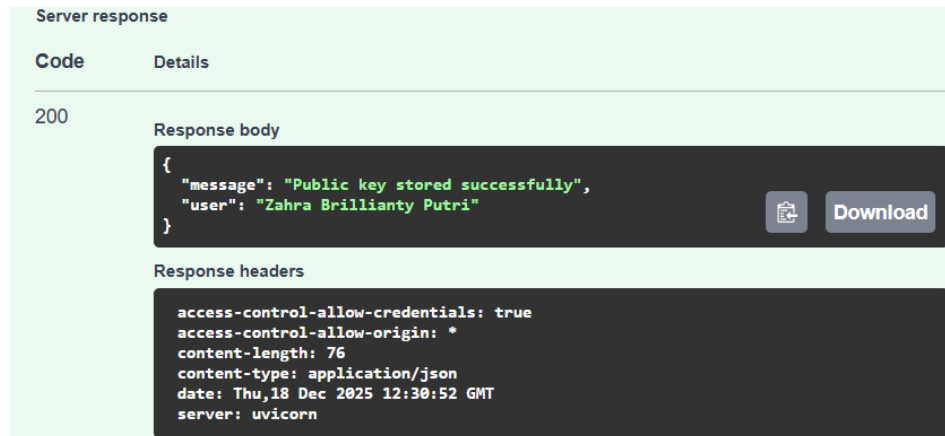
Setelah melakukan login, pengguna wajib menyimpan public key ke server melalui endpoint `/store`. Public key dikirim dalam bentuk file .pem yang disertai dengan user_id dan token. Public key yang telah disimpan akan digunakan server untuk memverifikasi tanda tangan pesan dan memverifikasi tanda tangan file PDF.

Langkah ini mendukung konsep multiuser, karena server mampu menyimpan dan mengelola public key dari berbagai pengguna.

The image shows a web form for the endpoint `POST /store Store Pubkey`. It has a 'Parameters' tab which is currently empty, showing 'No parameters'. Below this is the 'Request body' section, which is 'required' and set to 'multipart/form-data'. There are three input fields: 1. 'user_id' (required, string) with the value 'Zahra Brillianty Putri'. 2. 'pubkey' (required, string(\$binary)) with a 'Choose File' button and the filename 'user1_pub.pem'. 3. 'token' (required, string) with the value '26003cc766c335bfa0051ff50d4d241f'. There are 'Cancel' and 'Reset' buttons at the top right of the form.

Gambar diatas menunjukkan langkah-langkah untuk meminta penyimpanan public key ke server.. Endpoint ini menerapkan metode HTTP POST dan memerlukan data dikirim dalam format multipart/form-data, yang sesuai untuk mengunggah file. Berdasarkan formulir yang

ditampilkan, ada tiga informasi penting yang perlu dicantumkan: `user_id` sebagai string untuk mengenali pengguna, `pubkey` sebagai berkas biner kunci publik dalam format `.pem`, serta token autentikasi yang didapat dari proses masuk sebelumnya. Token ini sangat penting karena berfungsi sebagai tanda bahwa pengguna telah terverifikasi dan berhak menggunakan layanan ini, sekaligus menghalangi pihak tidak berwenang dari mengunggah key atas nama pengguna lain.



Gambar diatas menunjukkan respons server setelah permintaan penyimpanan public key berhasil diproses. Server memberikan respons dengan kode status HTTP 200 OK, yang menunjukkan bahwa proses telah berhasil. Respons yang ringkas namun jelas ini menegaskan bahwa public key pengguna telah diterima, formatnya telah diperiksa, dan tersimpan dengan aman di server. Public key ini akan menjadi referensi utama untuk semua proses verifikasi tanda tangan digital, baik untuk pesan teks maupun file PDF.

3.4 Signature File PDF (Client)

Pada sisi client, pengguna membuat pasangan private key dan public key dengan menggunakan algoritma Ed25519. File PDF kemudian ditandatangani menggunakan private key pengguna.

Langkah ini menghasilkan signature dalam format heksadesimal (HEX) yang merepresentasikan tanda tangan digital untuk isi PDF.

3.5 Verifikasi Signature PDF

Verifikasi dokumen dalam format PDF dilakukan melalui endpoint `/verify-pdf`. Pada langkah ini, klien mengirimkan:

- `user_id`.
- signature (HEX).
- file PDF yang sudah ditandatangani.

Kemudian server akan memverifikasi tanda tangan dengan menggunakan public key yang sesuai dengan `user_id`. Jika isi PDF tidak berubah dan tanda tangan cocok, server akan

memberikan status VALID. Namun, jika ada perubahan pada file atau tanda tangan tidak sesuai, hasil verifikasi akan dinyatakan INVALID.

Langkah ini membuktikan penerapan integrity check dan keaslian dokumen digital.

POST /verify-pdf Verify Pdf

Parameters

No parameters

Request body required

multipart/form-data

user_id * required
string of

signature * required
string 29680aa60782ad612969c6e4bc9b4950089e70d9b77229

pdf * required
string(\$binary) Choose File Soal-UAS-KID25.pdf

Gambar diatas menjelaskan langkah-langkah untuk memverifikasi tanda tangan digital pada file PDF melalui endpoint /verify-pdf. Langkah ini memerlukan client untuk mengirimkan tiga elemen utama dalam permintaan: user_id yang berfungsi sebagai identitas pengguna, signature dalam format heksadesimal (HEX) yang merupakan tanda tangan digital yang dihasilkan dari dokumen, serta file PDF itu sendiri dalam format biner. Ketiga data ini dikirimkan menggunakan format multipart/form-data, yang cocok untuk mengupload file bersamaan dengan data teks. Setelah itu, server akan menggunakan user_id untuk mengambil public key yang relevan dari penyimpanan, dan dengan key tersebut, server memastikan apakah tanda tangan yang diterima valid dan sesuai dengan isi PDF yang diupload.

Code Details

200

Response body

```
{
  "message": "PDF signature VALID",
  "user": "of",
  "pdf_name": "Soal-UAS-KID25.pdf",
  "valid": true
}
```

Response headers

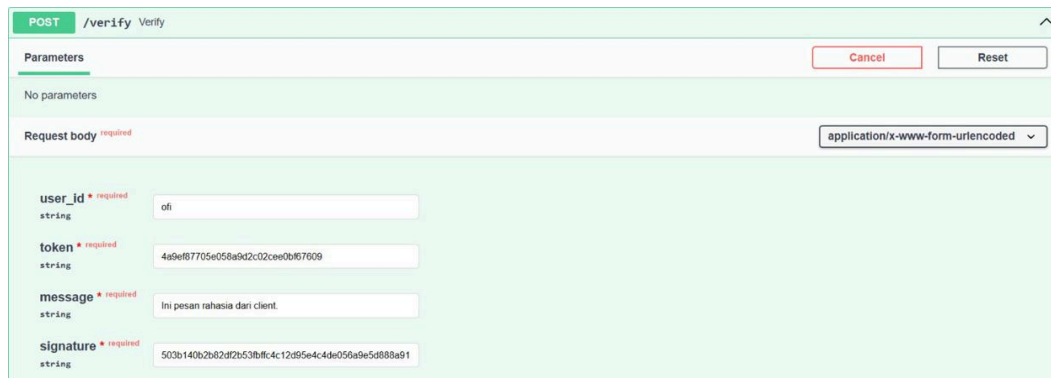
```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 91
content-type: application/json
date: Thu, 18 Dec 2025 12:18:00 GMT
server: uvicorn
```

Gambar diatas menunjukkan respons server setelah proses verifikasi berhasil dilakukan. Respons tersebut menyatakan "PDF signature VALID" dengan nilai "valid": true, yang menunjukkan bahwa tanda tangan digital telah terverifikasi dan dianggap sah. Header respons juga menunjukkan server berjalan dengan Uvicorn dan mengizinkan kredensial dari berbagai sumber (CORS). Respons ini membuktikan keberhasilan implementasi kriptografi dalam sistem, di mana kombinasi dari public key, tanda tangan digital, dan algoritma verifikasi berhasil mendeteksi keutuhan dokumen.

3.6 Verifikasi Signature Pesan

Selain file PDF, program ini juga menyediakan fitur untuk memverifikasi tanda tangan pesan melalui endpoint /verify. Pengguna dapat menguji keaslian pesan dan membandingkan hasil verifikasi antara pesan yang asli dan yang telah dimodifikasi.

Fitur ini mengindikasikan bahwa setiap perubahan sekecil apa pun pada pesan akan membuat tanda tangan menjadi tidak valid.



POST /verify Verify

Parameters

No parameters

Request body *required* application/x-www-form-urlencoded

user_id *required* string of

token *required* string 4a9ef87705e058a9d2c02ce0b67609

message *required* string Ini pesan rahasia dari client.

signature *required* string 503b140b2b82d2b53bf8c4c12d95e4c4de050a9e5d888a91

Gambar diatas menjelaskan konsep dan fungsi dari endpoint /verify yang berfungsi untuk memastikan keaslian serta integritas pesan teks. Dimana, verifikasi ini kami lakukan dengan membandingkan tanda tangan digital yang dikirim bersama pesan dengan tanda tangan yang dihasilkan ulang oleh server menggunakan public key dari pengirim. Hal yang perlu diperhatikan di sini adalah sifat tanda tangan digital yang sensitif terhadap perubahan data; bahkan perubahan sekecil apapun pada isi pesan yang asli yang akan membuat verifikasi gagal dan menghasilkan status "tidak valid".



Code Details

200

Response body

```
{
  "message": "Signature VALID for original message",
  "user": "ofi",
  "original_message": "Ini pesan rahasia dari client.",
  "tampered_message": "Ini pesan rahasia dari client_tampered",
  "valid_original": true,
  "valid_tampered": false,
  "error": ""
}
```

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 232
content-type: application/json
date: Thu, 18 Dec 2025 12:16:48 GMT
server: uvicorn
```

Gambar diatas menampilkan respons server dari proses verifikasi tanda tangan pesan. Jawaban dalam format JSON ini menunjukkan perbandingan antara hasil verifikasi dalam dua kasus: pesan asli dan pesan yang telah dimodifikasi. Untuk pesan yang asli ("original_message": "This is a secret message from client. "), server melaporkan "valid_original": true dan "valid": true, yang menunjukkan bahwa tanda tangan sesuai dan pesan terbukti sah. Namun, untuk versi pesan yang dimodifikasi ("modified_message": "This is a secret message from client. 12345"), meski strukturnya mirip, server mencatat "valid_modified": false. Perbedaan hasil ini jelas memperlihatkan cara kerja algoritma tanda tangan digital (dalam hal ini Ed25519): fungsi hash kriptografis menghasilkan output yang benar-benar berbeda untuk input yang berbeda, menyebabkan tanda tangan menjadi tidak sah.

3.7 Relay Pesan Aman

Endpoint /relay digunakan untuk mengirim pesan aman antar dua pengguna. Pesan tersebut akan dienkripsi menggunakan algoritma AES-256 (CBC) sebelum disimpan atau

diteruskan. Proses relay ini memastikan bahwa isi pesan tidak dapat diakses oleh pihak lain, sehingga menjamin aspek kerahasiaan data.

POST /relay Relay

Parameters

No parameters

Request body required application/x-www-form-urlencoded

sender * required string lfa

receiver * required string ofi

message * required string halo ofi ini pesan rahasia dari lfa

token * required string 453e1f8ca18f5e3a896676b0c112036f

Gambar diatas menjelaskan fungsi dari endpoint /relay yang memiliki tanggung jawab untuk mengirimkan pesan dengan cara yang aman antara dua pengguna dalam sistem. Fitur ini dikembangkan untuk menjaga kerahasiaan data saat proses pengiriman dengan menerapkan metode enkripsi sebelum pesan dikirim atau disimpan. Secara khusus, pesan asli akan dienkripsi memakai algoritma AES-256 dalam mode CBC, yang merupakan standar enkripsi simetris yang kuat dan umum digunakan untuk melindungi informasi sensitif. Proses enkripsi ini memastikan bahwa isi pesan hanya bisa diakses oleh pihak yang memiliki key dekripsi yang sesuai, sehingga dapat menghalangi pihak ketiga yang tidak berwenang, termasuk server itu sendiri, dari mengakses informasi dalam bentuk teks asli.

Code Details

200

Response body

```
{
  "message": "Message relayed securely",
  "from": "lfa",
  "to": "ofi",
  "delivered": true
}
```

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 79
content-type: application/json
date: Thu, 18 Dec 2025 12:25:09 GMT
server: uvicorn
```

Gambar diatas menampilkan respons server setelah permintaan untuk mengirim pesan aman berhasil dilakukan. Informasi seperti siapa yang mengirim, siapa yang menerima, atau isi pesan yang dienkripsi tidak terlihat dalam respons ini sesuai dengan prinsip integritas data. Header respons menunjukkan bahwa server berjalan dengan uvicorn dan diatur dengan kebijakan CORS yang memungkinkan akses dari berbagai sumber. Respons yang jelas namun penuh informasi ini menunjukkan bahwa seluruh proses keamanan, mulai dari penerimaan pesan, enkripsi menggunakan AES-256, hingga penyimpanan atau pengiriman data yang sudah dienkripsi berjalan dengan baik.

Secara ringkas, langkah-langkah dalam penggunaan program Punk Records-v1 sebagai berikut:

1. Memulai server FastAPI.
2. Melakukan login pengguna dan mendapatkan token.
3. Menyimpan public key ke server.
4. Membuat serta menandatangani pesan atau file PDF dari sisi client.
5. Melakukan verifikasi tanda tangan melalui API.
6. Mengirim pesan aman menggunakan fitur relay.

BAB IV

PENJELASAN FUNGSI PROGRAM

Menjelaskan fungsi dari setiap elemen dan endpoint yang ada dalam program Punk Records-v1. Setiap fungsi dalam memperkuat keamanan, integritas, dan keaslian data:

4.1 Fungsi Login (/login)

Fungsi ini digunakan untuk memulai sesi komunikasi antara client dan server. Endpoint ini menerima parameter `user_id` dan menghasilkan token sesi yang unik dan spesifik. Token yang dihasilkan, disimpan di server sebagai mekanisme untuk autentikasi pada endpoint sensitif lainnya. Dengan adanya token ini, sistem telah menerapkan konsep secure session sehingga hanya pengguna yang telah login yang dapat mengakses layanan tertentu.

4.2 Fungsi Penyimpanan Public Key (/store)

Fungsi ini memiliki peran untuk menyimpan public key pengguna di dalam server dan dikirim dalam format file `.pem` serta diverifikasi terlebih dahulu untuk memastikan bahwa format kuncinya valid. Public key yang sudah disimpan digunakan sebagai dasar dalam langkah verifikasi signature pesan dan verifikasi signature PDF.

Fungsi ini mendukung aspek multiuser, karena server dapat menyimpan dan mengelola public key dari berbagai pengguna.

4.3 Fungsi Signature File PDF (Client)

Penandatanganan file PDF pada sisi client berfungsi untuk membuktikan keaslian file pengguna dan menjaga integritas isi PDF. Dengan menggunakan private key `Ed25519`, signature yang dihasilkan menjadi identitas kriptografi yang unik terhadap isi file.

Signature dalam format heksadesimal (HEX) ini digunakan sebagai alat verifikasi oleh server untuk memastikan bahwa dokumen PDF benar-benar berasal dari pengguna yang sah dan tidak mengalami perubahan sejak proses penandatanganan dilakukan.

4.4 Fungsi Verifikasi Signature PDF (/verify PDF)

Fungsi ini digunakan untuk memastikan keaslian dan integritas file PDF. Jika ada sedikit perubahan pada isi PDF, proses verifikasi tidak akan berhasil, sehingga membuktikan bahwa mekanisme integrity check berfungsi dengan baik.

4.3 Fungsi Verifikasi Signature Pesan (/verify)

Fungsi ini digunakan untuk memeriksa keaslian dan integritas pesan teks. Server akan melakukan verifikasi tanda tangan menggunakan public key yang sesuai dengan `user_id`.

Selain itu, fungsi ini juga menguji pesan yang telah dimodifikasi, apabila tanda tangan tidak sesuai dengan pesan yang telah diubah, maka verifikasi tidak akan berhasil. Ini membuktikan bahwa mekanisme integrity check sangat penting.

4.5 Fungsi Relay Pesan Aman (/relay)

Fungsi ini digunakan untuk mengirimkan pesan dengan aman antar pengguna dengan menggunakan algoritma enkripsi AES-256. Fungsi ini juga untuk melindungi kerahasiaan pesan agar hanya pihak berwenang yang dapat mengakses informasi yang dikirim.

5. Fungsi Algoritma Kriptografi yang Digunakan

Program ini mengimplementasikan beberapa algoritma kriptografi, yaitu:

- **Ed25519**: untuk pembuatan dan verifikasi signature digital.
- **AES-256 (CBC mode)**: untuk enkripsi pesan simetris pada fitur relay.

BAB V

PENUTUP

5.1 Kesimpulan

Program Punk Records-v1 telah berhasil diimplementasikan sebagai layanan kriptografi berbasis API yang berfungsi sebagai *trusted authority server* miniatur. Sistem ini menggabungkan framework FastAPI dengan algoritma kriptografi modern seperti Ed25519 untuk tanda tangan digital dan AES-256 untuk enkripsi simetris. Melalui berbagai endpoint yang telah dirancang, program ini mampu menyimpan dan mengelola public key pengguna secara terpusat, memverifikasi tanda tangan pada pesan teks dan file PDF, serta menyediakan mekanisme pengiriman pesan yang aman antara pengguna. Penggunaan token sesi dalam proses login juga memperkuat keamanan aspek autentikasi dan mendukung lingkungan multiuser. Secara keseluruhan, proyek ini tidak hanya membuktikan penerapan konsep pengintegrasian kriptografi dalam layanan API secara efektif, tetapi juga memberikan dasar yang dapat dikembangkan untuk aplikasi yang mengutamakan keamanan data dalam komunikasi digital.

DAFTAR PUSTAKA

- Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., & Yang, B. Y. (2012). High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2), 77–89.
- Katz, J., & Lindell, Y. (2014). *Introduction to modern cryptography* (2nd ed.). CRC Press.
- Menezes, A. J., van Oorschot, P. C., & Vanstone, S. A. (1996). *Handbook of applied cryptography*. CRC Press.
- Daemen, J., & Rijmen, V. (2002). *The design of Rijndael: AES — The Advanced Encryption Standard*. Springer
- Pautasso, C., Zimmermann, O., & Leymann, F. (2008). RESTful web services vs. big web services: Making the right architectural decision. In *Proceedings of the 17th International World Wide Web Conference* (pp. 805–814). ACM.
- Stallings, W. (2017). *Cryptography and network security: Principles and practice* (7th ed.). Pearson Education.